# EXPERIMENT – 11

## Performing IP Spoofing Using a GitHub Tool in Kali Linux (Educational Demonstration)

### Aim

To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

### Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Tool Source:** GitHub (IP Spoofing Tool – Educational Use)
- **Programming Language:** Python / Bash (as per tool)
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

### Introduction

IP spoofing is a technique where an attacker forges the source IP address in network packets to impersonate another system. It is commonly used in attacks such as DDoS, session hijacking, and bypassing IP-based authentication. This experiment demonstrates IP spoofing using a publicly available GitHub tool to understand how attackers manipulate packet headers. Kali Linux provides a safe and controlled environment for ethical network security experimentation.

### Procedure

### Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

### Step 2: Open Terminal

1. Launch Terminal in Kali Linux.
2. Update system packages:
3. sudo apt update

### Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:

2. git clone https://github.com/<username>/<ip-spoofing-tool>
3. Navigate to the tool directory:
4. cd ip-spoofing-tool

**Step 4: Install Dependencies**

1. Install required dependencies:
2. sudo apt install python3
3. pip3 install -r requirements.txt

**Step 5: Configure the Tool**

1. Open the script file:
2. nano spoof.py
3. Configure:
    - Target IP address
    - Spoofed source IP address
    - Network interface

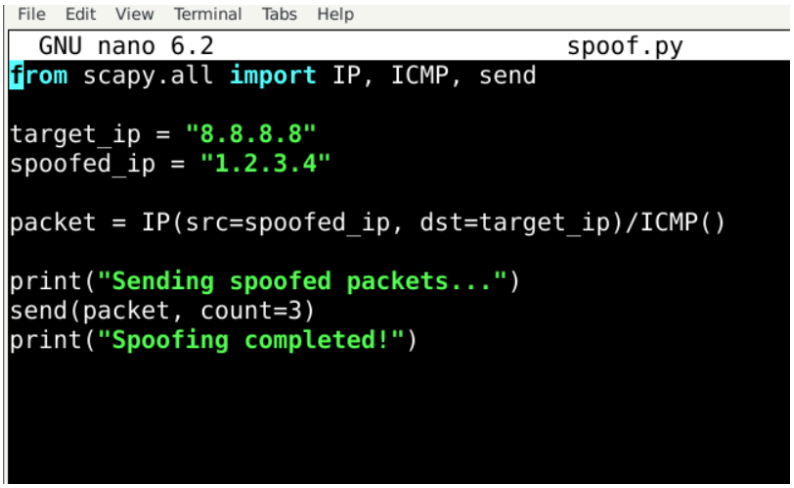
**Step 6: Execute IP Spoofing**

1. Run the tool with root privileges:
2. sudo python3 spoof.py
3. Observe packet transmission with spoofed IP addresses.

**Step 7: Monitor Network Traffic (Optional)**

1. Use Wireshark or tcpdump:
2. sudo tcpdump
3. Verify spoofed source IP packets.

**Output**



```
└─$ sudo tcpdump -i eth0
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^[[A^
^C
0 packets captured
0 packets received by filter
0 packets dropped by kernel
```

```
 id 35488, seq 18, length 64
09:17:01.376699 IP tzdelb-bf-in-f14.1e100.net > 10.0.2.15: ICMP echo reply, i
d 35488, seq 18, length 64
09:17:02.295940 IP 10.0.2.15 > tzdelb-bf-in-f14.1e100.net: ICMP echo request,
 id 35488, seq 19, length 64
09:17:02.366752 IP tzdelb-bf-in-f14.1e100.net > 10.0.2.15: ICMP echo reply, i
d 35488, seq 19, length 64
09:17:03.297960 IP 10.0.2.15 > tzdelb-bf-in-f14.1e100.net: ICMP echo request,
 id 35488, seq 20, length 64
09:17:03.377280 IP tzdelb-bf-in-f14.1e100.net > 10.0.2.15: ICMP echo reply, i
d 35488, seq 20, length 64
09:17:04.299345 IP 10.0.2.15 > tzdelb-bf-in-f14.1e100.net: ICMP echo request,
 id 35488, seq 21, length 64
09:17:04.367626 IP tzdelb-bf-in-f14.1e100.net > 10.0.2.15: ICMP echo reply, i
d 35488, seq 21, length 64
09:17:05.305258 IP 10.0.2.15 > tzdelb-bf-in-f14.1e100.net: ICMP echo request,
 id 35488, seq 22, length 64
09:17:05.386239 IP tzdelb-bf-in-f14.1e100.net > 10.0.2.15: ICMP echo reply, i
d 35488, seq 22, length 64
09:17:06.307130 IP 10.0.2.15 > tzdelb-bf-in-f14.1e100.net: ICMP echo request,
 id 35488, seq 23, length 64
09:17:06.376639 IP tzdelb-bf-in-f14.1e100.net > 10.0.2.15: ICMP echo reply, i
d 35488, seq 23, length 64
09:17:07.308228 IP 10.0.2.15 > tzdelb-bf-in-f14.1e100.net: ICMP echo request,
 id 35488, seq 24, length 64
09:17:07.373431 IP tzdelb-bf-in-f14.1e100.net > 10.0.2.15: ICMP echo reply, i
d 35488, seq 24, length 64
```

```
5 time=200 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=12 ttl=25
5 time=67.2 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=13 ttl=25
5 time=257 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=14 ttl=25
5 time=66.1 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=15 ttl=25
5 time=65.4 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=16 ttl=25
5 time=64.0 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=17 ttl=25
5 time=65.1 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=18 ttl=25
5 time=83.6 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=19 ttl=25
5 time=70.8 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=20 ttl=25
5 time=79.3 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=21 ttl=25
5 time=68.3 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=22 ttl=25
5 time=81.0 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=23 ttl=25
5 time=69.5 ms
64 bytes from tzdelb-bf-in-f14.1e100.net (142.250.182.14): icmp_seq=24 ttl=25
5 time=65.3 ms
```

**Result**

IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux.
The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.